ryukih Update README.md 1dc8b5b · 3 days ago History

Name	Last commit message	Last commit date
..		
visualization	up	3 days ago
.gitignore	create on-fire	3 weeks ago
CLAUDE.md	up	4 days ago
README.md	Update README.md	3 days ago
agent.py	up	4 days ago
config.yaml	up	4 days ago
main.py	up	4 days ago
ollama_client.py	create on-fire	3 weeks ago
requirements.txt	create on-fire	3 weeks ago
setup_mac.sh	up	4 days ago
setup_win.bat	up	4 days ago
simulation.py	up	4 days ago
utils.py	up	4 days ago
visualization.py	up	4 days ago

README.md

LLM Multi-Agent 2D Simulation

概要

LLMエージェントの創発性と自律性を発現させることを目的とした2次元空間マルチエージェントシミュレーション。エージェントには生の数値データ（占有率、火災の強度・距離等）のみを提供し、行動指示や定性的評価（「危険」「快適」等）は一切与えない。エージェント自身が状況を解釈し、意思決定・コミュニケーションを通じて集団的な行動パターンが創発する様子を観察する。

フィールドに複数の場所（デフォルトでは左側と右側にバーが1つずつ）が存在し、各場所は独立した収容上限（capacity）を持つ。シミュレーション途中で火災イベントも発生する。

simulation.mp4

デモ動画：火災近傍では、単純な退避にとどまらず、合流指示・物資確保・他者待機・監視などの創発的な判断が観察された。

Fire Event（火事イベント）

シミュレーションの途中で、指定位置（またはランダムな位置）に複数の火事を発生させることができます。火事には以下の特徴があります:

- **知覚モデルB（距離依存）**: 火事の知覚半径（`radius`）内にいるエージェントのみが火事情報を直接受け取る。半径外のエージェントはプロンプトに火事情報が一切含まれず、他エージェントからのメッセージ経由でのみ間接的に知る
- **定量情報のみ**: エージェントに与えられるのは火事の位置、強度（0.0-1.0）、半径、自分との距離のみ。「危険」「避難すべき」等の定性的な記述は一切含まれない
- **時間不変**: 火事の強度・位置は発生後変化しない
- **発生前は無情報**: 火事が発生するステップより前のプロンプトには、火事に関するセクションは存在しない

この設計により、エージェントが定量情報のみからどのような行動（回避、情報伝達、無視等）を創発的に生み出すかを観察できます。

各エージェントには**ペルソナ（性別: male/female）**がランダムに割り当てられ、LLMプロンプトや近隣エージェント情報に性別が含まれます。エージェントは相手の性別を認識した上でコミュニケーションや意思決定を行います。

必要な環境

- Python 3.8以上
- Ollama（LLMサーバー）
- 必要なPythonパッケージ（requirements.txt参照）

セットアップ

1. 仮想環境の作成とセットアップ

macOS/Linux の場合:

```
# セットアップスクリプトを実行（推奨）
chmod +x setup_mac.sh
./setup_mac.sh

# または手動でセットアップ
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Windows の場合:

```
REM セットアップスクリプトを実行（推奨）
setup_win.bat

REM または手動でセットアップ
python -m venv venv
venv\Scripts\activate.bat
python -m pip install --upgrade pip
pip install -r requirements.txt
```

2. 仮想環境の有効化

macOS/Linux:

```
source venv/bin/activate
```

Windows:

```
venv\Scripts\activate.bat
```

3. Ollamaのセットアップ

Ollamaのインストール

- Ollamaをインストール

Ollamaサーバーの起動

Ollamaは通常、インストール時に自動的にバックグラウンドサービスとして起動します。しかし、手動で起動する必要がある場合や、APIサーバーとして明示的に起動したい場合は、以下の手順を実行してください:

方法1: 別のターミナルで手動起動（推奨）

```
# 新しいターミナルを開いて実行
ollama serve
```

これにより、デフォルトで `http://localhost:11434` でサーバーが起動します。

方法2: バックグラウンドで起動（macOS/Linux）

```
ollama serve &
```

Ollamaサーバーの状態確認

```
# サーバーが起動しているか確認
curl http://localhost:11434/api/tags
```

正常に起動していれば、モデルリストがJSON形式で返されます。

モデルのダウンロードと選択

利用可能なモデルの確認:

```
# Ollamaにダウンロード済みのモデル一覧を表示
ollama list
```

モデルのダウンロード:

```
# 使用するモデルをダウンロード（例: llama3.2）
ollama pull llama3.2

# または他のモデルを使用する場合
ollama pull llama3.1
ollama pull mistral
ollama pull qwen2.5
```

モデルの選択:

- シミュレーションは `config.yaml` の `llm.model` で指定されたモデルを使用します
- デフォルトは `llama3.2` です
- 異なるモデルを使用する場合は、 `config.yaml` を編集してください:

```
llm:
  model: "qwen2.5:latest"  # 使用したいモデル名に変更
```

モデル名の指定方法:

- **推奨:** `ollama list` で表示される形式をそのまま使用（例: `qwen2.5:latest`）
- **省略可能:** タグが `:latest` の場合は、タグを省略しても動作します（例: `qwen2.5`）
- **例:**
  - `qwen2.5:latest` ✅ （推奨）
  - `qwen2.5` ✅ （ `:latest` を省略）
  - `llama3.2:latest` ✅
  - `llama3.2` ✅
  - `gpt-oss:20b` ✅ （特定のタグを指定）

注意:

- `config.yaml` の `llm.model` で指定したモデル名と一致するモデルが使用されます
- 指定したモデルが存在しない場合、シミュレーション開始時にエラーメッセージが表示されます
- 複数のモデルがダウンロードされていても、 `config.yaml` で指定した1つのモデルのみが使用されます
- `ollama list` で表示される `NAME` 列の値をそのまま使用するのが最も確実です

4. 設定ファイルの確認

- `config.yaml` でパラメータを確認・調整

使用方法

注意:

- 実行前に仮想環境を有効化してください
- Ollamaサーバーが起動していることを確認してください（ `ollama serve` を別のターミナルで実行）

基本的な実行

```
# 仮想環境を有効化（まだ有効化していない場合）
source venv/bin/activate  # macOS/Linux
# または
venv\Scripts\activate.bat  # Windows

# シミュレーションを実行
python main.py
```

可視化を有効にする

注意: WSL環境やGUIが利用できない環境では、 `--visualize` オプションは動作しません。代わりに `--save-frames` を使用してください。

```
python main.py --visualize
```

フレームを保存する

```
python main.py --save-frames
```

カスタム設定ファイルを使用

```
python main.py --config custom_config.yaml
```

設定ファイル（config.yaml）

主要なパラメータ:

- **simulation**: シミュレーション設定
  - `duration` : シミュレーションステップ数
  - `half_space_size` : 空間の半分のサイズ（例: 25 → 座標範囲は -25 ～ +25）
  - `half_place_size` : 場所の半分のサイズ（各場所の `half_size` が優先されます）
- **agents**: エージェント設定
  - `num_agents` : エージェント数
  - `communication_radius` : 通信半径
  - `memory_limit` : 保存する最大メモリ数（デフォルト: 20）
  - `memory_size` : LLM推論に使用するメモリ数（デフォルト: 5）
  - `message_history_limit` : 保存する最大メッセージ数（デフォルト: 10）
  - `message_context_size` : LLM推論に使用するメッセージ数（デフォルト: 3）
- **places**: 場所設定（複数場所対応、将来的にカフェや図書館などにも拡張可能）
  - 場所のリスト。各場所は以下の設定を持ちます:
    - `name` : 場所名（必須、例: "left_bar", "right_bar", "cafe", "library"）
    - `type` : 場所の種類（必須、例: "bar", "cafe", "library"）
    - `center_x` : 場所の中心X座標（必須）
    - `center_y` : 場所の中心Y座標（必須）
    - `half_size` : 場所の半分のサイズ（必須、場所は中心から±half_sizeの範囲）
    - `capacity` : その場所の収容上限人数（必須、占有率の計算に使用）

  例（現在の設定）：

```
places:
  - name: "left_bar"
    type: "bar"  # 場所の種類: bar, cafe, library など
    center_x: -15
    center_y: 0
    half_size: 5
    capacity: 12
  - name: "right_bar"
    type: "bar"
    center_x: 15
    center_y: 0
    half_size: 5
    capacity: 10
```

  将来の拡張例（カフェや図書館など）：

```
places:
  - name: "cafe"
    type: "cafe"
    center_x: -15
    center_y: 0
    half_size: 5
    capacity: 12
  - name: "library"
    type: "library"
    center_x: 15
    center_y: 0
    half_size: 5
    capacity: 10
```

- **fires**: 火事イベント設定（複数火事対応）
  - 火事のリスト。各火事は以下の設定を持ちます:
    - `name` : 火事の名前（必須、例: "fire_1"）
    - `start_step` : 火事が発生するステップ（必須）
    - `intensity` : 火事の強度（0.0〜1.0、エージェントには定量値としてのみ伝達）
    - `radius` : 知覚半径（この距離内のエージェントのみ火事情報を直接受け取る）
    - `center_x` : 火事の中心X座標（省略時はランダム位置）
    - `center_y` : 火事の中心Y座標（省略時はランダム位置）

```
fires:
  - name: "fire_1"
    start_step: 15
    intensity: 0.8
    radius: 10
    center_x: 0         # 省略するとランダム位置
    center_y: 0
  - name: "fire_2"
    start_step: 30
    intensity: 0.5
    radius: 8
    center_x: -10
    center_y: 10
```

- **llm**: LLM設定（詳細は `config.yaml` を参照）
  - `model` : Ollamaモデル名
  - `base_url` : Ollama APIエンドポイント
  - `temperature` : サンプリング温度（低いほど一貫した行動、高いほど多様な行動）
  - `max_tokens` : 最大トークン数
  - `repeat_penalty` : 反復ペナルティ（同じトークンの繰り返しを抑制）
  - `repeat_last_n` : 反復チェック範囲（繰り返し検出の対象トークン数。 `0` で無効、 `-1` で `num_ctx` を使用）
  - `min_p` : Min-Pサンプリング（指定確率未満のトークンを除外）

座標系について

このシミュレーションは**原点中心の座標系**を使用します:

- **フィールド**: 原点 (0, 0) を中心に、 `-half_space_size` 〜 `+half_space_size` の範囲
- **場所**: 各場所は独立した中心位置（ `center_x` , `center_y` ）を持ち、その中心から `±half_size` の範囲（両端を含む）

エージェントの知識:

- エージェントは**すべての場所の位置情報**を知っています（プロンプトに含まれます）
- ただし、場所の占有状況（エージェント数、収容上限、占有率）は、**その場所内にいるエージェントのみ**が直接受け取ります

モデルの選択について:

- `config.yaml` の `llm.model` で使用するモデルを指定
- 利用可能なモデルを確認するには: `ollama list`
- モデルをダウンロードするには: `ollama pull <モデル名>`

出力

- `output/` : 可視化フレームと統計グラフが保存されます
  - **可視化では、エージェントの性別を色（男=青、女=赤）、場所内/外をマーカー形状（場所内=★、場所外=●）で表現**
  - 火事発生後は各火事の**火事中心（赤三角）と知覚半径（破線円）が描画される（複数火事対応）。火事の範囲はintensityに応じたカラーマップ（YlOrRd）で塗られ、右側に常時表示されるカラーバー（0.0〜1.0、"Intensity of Fire"）**で強度を確認できる
  - 統計グラフには「火事半径内エージェント数の時系列」サブプロットが追加され、各火事の発生ステップに垂直線が描画される
- `output/messages.jsonl` : エージェント間メッセージ履歴（火事情報の伝播パターンの分析に有用）
- `output/memory_reasoning.jsonl` : エージェントの記憶と推論ログ
- `simulation.log` : シミュレーションログ

LLMエージェント設計

基本設計

- 各エージェントは毎ステップ、**Message / Memory / Action** をLLMから生成し、同期的に行動
- 同一のLLMエンジンを全エージェントで共有し、差分は各自のMemoryと相互作用履歴のみ（個性は履歴から創発）
- プロンプトは「最適化タスク」を明示せず、状況説明＋数値データ＋近傍メッセージ＋自己状態を与える構成

ペルソナ（性別）

- 各エージェントには初期化時に **male** または **female** がランダム（1/2の確率）で割り当てられる
- LLMプロンプトの冒頭（ `You are Agent N (male/female)` ）および自己状態欄（ `Gender:` ）に性別が含まれる
- 近隣エージェント情報にも相手の性別が表示される（例: `Agent 3 (female) is outside the places` ）
- エージェントは相手の性別を認識した上でメッセージ内容や行動を決定する

Action（行動）

4方向の離散選択 + 滞在:

- `up` : Y+1（上方向）
- `down` : Y-1（下方向）
- `left` : X-1（左方向）
- `right` : X+1（右方向）
- `stay` : 現在位置に滞在

Memory（記憶）

- LLMが出力した `memory` フィールドが次ステップの「Previous Memory」として自己フィードバック
- 内部状態が履歴依存で進化し、個性が創発する仕組み
- `memory_limit` : 保存する最大メモリ数（古いものから削除）
- `memory_size` : LLM推論時に参照する直近のメモリ数

場所内限定情報

各場所内のエージェントのみが以下の数値データを直接受け取る:

- 現在のエージェント数（Number of agents here）
- 収容上限（Capacity）
- 占有率（Occupancy rate = エージェント数 / 収容上限）

定性的な評価（快適/不快等）は一切含まれず、数値の解釈はエージェント自身に委ねられる。

場所外のエージェントはこれらの情報を受け取らない（会話や推論で間接的に学ぶ）

注意: エージェントはすべての場所の位置情報を知っていますが、各場所の占有状況はその場所内にいるエージェントのみが知ることができます。

コミュニケーション

- 通信半径内（デフォルト: 5セル）のエージェント間でメッセージ交換が可能
- **同一領域条件**: 以下の条件を満たすエージェント間でのみ通信可能
  - 両方とも場所外にいる、または
  - 両方とも同じ場所内にいる

  通信制限:

  - ❌ 場所内のエージェント ↔ 場所外のエージェント: 通信不可
  - ❌ 異なる場所内のエージェント同士: 通信不可
  - ✅ 同じ場所内のエージェント同士: 通信可能
  - ✅ 両方とも場所外のエージェント同士: 通信可能
- `message_history_limit` : 保存する最大メッセージ数（古いものから削除）
- `message_context_size` : LLM推論時に参照する直近のメッセージ数

シミュレーションステップの実行順序

各ステップは以下の順序で実行されます:

0. **火事活性化チェック** - 各火事について `step >= start_step` の場合、指定位置（またはランダム位置）に火事を発生（各火事1回のみ）
1. **Phase 1: メッセージ決定** - 全エージェントがLLMでメッセージを決定（いずれかの火事の知覚半径内のエージェントには該当する火事情報を含む）
2. **Phase 2: メッセージ送信** - 意思決定時点での近傍エージェントにメッセージを送信
3. **Phase 3: 行動決定** - 全エージェントがLLMで行動（move/stay）を決定（火事情報を含む）
4. **Phase 4: 移動実行** - エージェントが決定した方向に移動

この順序により、メッセージは移動前の位置関係に基づいて送信されます。

LLM出力形式

メッセージ決定（Phase 1）：

```
{
    "message": "近傍エージェントへのメッセージ（任意、最大200語）",
    "reasoning": "メッセージ送信の理由"
}
```

行動決定（Phase 3）：

```
{
    "action": "move" or "stay",
    "direction": "up", "down", "left", or "right",
    "memory": "次ステップのために記憶したいこと",
    "reasoning": "決定理由"
}
```

ライセンス

作成者: 兵頭龍樹（Ph.D.）

このコードは研究目的で提供されています。